

Bài tập Chọn kiến trúc cho hệ thống đặt lịch dịch vụ (SmartBooking)

Thông tin sinh viên

- **Họ và tên:** Lê Thiên Phú
- **MSSV:** 23127244

Phần A - Chọn kiến trúc tổng thể

Em/nhóm em chọn kiến trúc: Modular Monolith

1. Vì sao chọn kiến trúc này?

SmartBooking hiện chỉ có 5 lập trình viên và 1 DevOps bán thời gian, cần ra sản phẩm đầu tiên trong 3 tháng, với quy mô dự kiến khoảng 10.000 người dùng trong 6 tháng đầu - đây là quy mô vừa phải, chưa phải bài toán cần scale cực lớn. Modular Monolith cho phép nhóm code, test, deploy như một ứng dụng duy nhất (nhánh, đơn giản, một pipeline CI/CD), nhưng vẫn chia module theo ranh giới nghiệp vụ rõ ràng (Người dùng, Dịch vụ, Đặt lịch, Thanh toán, Thông báo, Nhân viên, Báo cáo). Nhờ vậy team vẫn kịp deadline 3 tháng mà không phải trả giá bằng một "big ball of mud" như Monolith thường gặp khi phình to.

2. Vì sao chưa chọn hai kiến trúc còn lại?

- **Monolith (không chia module rõ ràng):** dễ làm nhanh nhất lúc đầu, nhưng khi hệ thống lớn lên (mở thêm chi nhánh sau 1 năm) sẽ rất khó tách vì code các phần đan xen nhau, không có ranh giới rõ để cắt ra.
- **Microservices:** đúng là linh hoạt, dễ scale từng phần, cách ly lỗi tốt - nhưng đi kèm chi phí vận hành lớn: nhiều pipeline CI/CD, service discovery, giám sát phân tán, xử lý giao dịch phân tán (Saga thay vì transaction ACID)... Với chỉ 1 DevOps bán thời gian và 5 dev, team chưa đủ nhân lực để vận hành một hệ thống phân tán thực sự ngay từ bản đầu tiên. Theo tài liệu kiến trúc microservices, microservices phù hợp khi "tổ chức có nhiều khả năng kinh doanh độc lập" và "nhóm đã có kinh nghiệm vận hành hệ thống phân tán" - SmartBooking ở giai đoạn này chưa đáp ứng các điều kiện đó, nên chọn microservices ngay là rủi ro không cần thiết ("chạy theo xu hướng mà không đánh giá kỹ").

3. Kiến trúc này có phù hợp với đội 5 lập trình viên và 1 DevOps bán thời gian không?

Có. Modular Monolith chỉ cần một hạ tầng deploy (1 server/container, 1 database ban đầu), không cần đội DevOps túc trực để quản lý hàng chục service như microservices. 5 dev có thể chia nhau phụ trách từng module (vd 1-2 người/module) mà không đụng code nhau nhờ ranh giới module rõ ràng, vẫn giữ được tốc độ phát triển nhanh cho deadline 3 tháng.

4. Nếu hệ thống lớn lên sau 1 năm, phần nào có thể tách ra trước?

Hai module có khả năng tách ra sớm nhất:

- **Thanh toán (Payment):** vì tích hợp với cổng thanh toán bên thứ ba, có yêu cầu bảo mật/tuân thủ riêng, và tải xử lý có đặc thù khác các module còn lại (cần retry, đối soát...). Tách ra giúp cô lập rủi ro và scale riêng khi giao dịch tăng.

- **Thông báo (Notification):** vì đây vốn đã là tác vụ bất đồng bộ (gửi email/SMS), không cần chia sẻ transaction với phần lõi, tách sớm gần như không rủi ro và dễ scale ngang khi có thêm chi nhánh/lượng thông báo tăng.

Khi công ty mở thêm chi nhánh, module **Báo cáo (Report)** cũng là ứng viên tách tiếp theo, vì lúc đó dữ liệu tổng hợp từ nhiều chi nhánh sẽ cần mô hình đọc riêng (read model), tách khỏi luồng ghi giao dịch chính để không ảnh hưởng hiệu năng đặt lịch.

Phần B - Chia chức năng hệ thống

Phần chức năng	Nhiệm vụ chính	Có nên tách riêng ngay không? Vì sao?
Người dùng	Quản lý tài khoản, thông tin khách hàng	Không. Là dữ liệu lõi được nhiều module khác dùng chung (Đặt lịch, Nhân viên). Tách sớm sẽ tạo thêm một API/network call cho gần như mọi request, không cần thiết ở quy mô 10.000 user.
Dịch vụ	Quản lý danh sách dịch vụ	Không. Dữ liệu ít thay đổi, tải đọc thấp, gắn chặt với luồng đặt lịch (chọn dịch vụ → chọn giờ). Giữ chung giúp query nhanh, tránh join xuyên service.
Đặt lịch	Tạo và quản lý lịch hẹn	Không tách ngay , nhưng là module trung tâm cần thiết kế ranh giới (module boundary) rõ ràng nhất ngay từ đầu, vì đây là nghiệp vụ lõi và là module đầu tiên có khả năng cần tách khi tải tăng cao.
Thanh toán	Xử lý giao dịch thanh toán	Nên thiết kế tách biệt rõ ràng nhất (module riêng, sẵn sàng thành service khi cần) vì tích hợp cổng thanh toán ngoài, có yêu cầu bảo mật/tuân thủ (PCI-DSS) khác hẳn phần còn lại, và là nơi dễ phát sinh lỗi cần cô lập.
Thông báo	Gửi email/SMS	Nên xử lý bất đồng bộ ngay từ đầu (module event-driven riêng) , vì đây là tác vụ phụ trợ, không ảnh hưởng kết quả chính người dùng thấy ngay, và là phần dễ tách thành service độc lập nhất về sau (ít rủi ro, không giữ dữ liệu giao dịch).
Nhân viên	Xem và xác nhận lịch	Không. Chỉ là một góc nhìn (view) và quyền hạn khác trên cùng dữ liệu Đặt lịch, tách riêng sẽ phải đồng bộ dữ liệu hai nơi không cần thiết.
Báo cáo	Thống kê số lượng lịch đặt, doanh thu	Không tách ngay , nhưng nên tách khi mở rộng nhiều chi nhánh, vì lúc đó khối lượng đọc/tổng hợp dữ liệu lớn có thể ảnh hưởng hiệu năng ghi của luồng đặt lịch chính.

Lưu ý: Với kiến trúc Modular Monolith, **không có phần nào trở thành microservice riêng ngay ở bản đầu tiên** - tất cả vẫn nằm chung một ứng dụng, một database. Điều được tách là **ranh giới module trong code** (mỗi module một package/namespace, giao tiếp qua interface nội bộ rõ ràng, không truy cập trực tiếp bảng dữ liệu của module khác), để khi cần tách thật sự (Thanh toán, Thông báo là ưu tiên) thì việc bóc ra thành service riêng không phải viết lại từ đầu.

Phần C - Luồng đặt lịch

Luồng đặt lịch:

Người dùng chọn dịch vụ
 → chọn ngày giờ
 → hệ thống kiểm tra còn lịch trống
 → tạo booking tạm (Pending)
 → thanh toán
 → xác nhận booking (Confirmed)
 → gửi email/SMS
 → cập nhật báo cáo

Bước xử lý	Nên xử lý ngay hay xử lý bằng sự kiện?	Lý do
Kiểm tra lịch trống	Xử lý ngay	Ảnh hưởng trực tiếp đến kết quả người dùng thấy tức thì (còn slot hay không); cần đồng bộ để tránh double-booking khi nhiều người chọn cùng giờ.
Tạo booking	Xử lý ngay	Là bước lỗi của giao dịch, cần trả về bookingId ngay để người dùng tiếp tục sang bước thanh toán.
Thanh toán	Xử lý ngay	Người dùng cần biết ngay giao dịch thành công hay thất bại để quyết định bước tiếp theo (thử lại thẻ khác, hủy...); không thể để "chờ sự kiện" mới báo kết quả.
Gửi email/SMS	Xử lý bằng sự kiện	Không ảnh hưởng đến kết quả chính (booking đã xong), có thể trễ vài giây/phút không sao; tách ra để không làm chậm phản hồi cho người dùng và tránh lỗi bên thứ ba (SMS gateway chậm) làm ảnh hưởng giao dịch chính.
Cập nhật báo cáo	Xử lý bằng sự kiện	Là tác vụ tổng hợp, không cần chính xác real-time tuyệt đối; xử lý bất đồng bộ giúp giảm tải cho luồng đặt lịch chính.
Ghi log/phân tích dữ liệu	Xử lý bằng sự kiện	Thuần phụ trợ, hoàn toàn nằm ngoài "critical path" của người dùng.

Theo tài liệu EDA: những việc ảnh hưởng trực tiếp đến kết quả người dùng nhìn thấy (kiểm tra chỗ trống, tạo booking, thanh toán) nên xử lý ngay (đồng bộ); những việc phụ trợ (thông báo, báo cáo, log) phát sự kiện để các thành phần khác tự tiêu thụ - giúp giảm phụ thuộc (loose coupling) và luồng chính phản hồi nhanh hơn.

Phần D - Dùng sự kiện như thế nào?

Sau khi đặt lịch thành công, hệ thống phát ra sự kiện **BookingConfirmed**, và các sự kiện liên quan khác trong luồng:

Sự kiện	Ai phát ra?	Ai nhận?	Nhận để làm gì?
BookingConfirmed	Booking	Notification	Gửi email/SMS xác nhận lịch hẹn
BookingConfirmed	Booking	Report	Cập nhật số liệu booking thành công
BookingConfirmed	Booking	Staff	Hiển thị lịch mới cho nhân viên xác nhận
PaymentSucceeded	Payment	Booking	Chuyển booking từ trạng thái Pending sang Confirmed , phát tiếp BookingConfirmed
PaymentSucceeded	Payment	Report	Ghi nhận doanh thu vào báo cáo
PaymentFailed	Payment	Booking	Hủy booking tạm (giải phóng slot đã giữ), phát tiếp BookingCancelled
PaymentFailed	Payment	Notification	Gửi thông báo cho khách biết thanh toán thất bại, mời thử lại
BookingCancelled	Booking	Report	Cập nhật số liệu lịch bị hủy
BookingCancelled	Booking	Notification	Gửi email/SMS thông báo lịch đã bị hủy

Ba sự kiện thêm theo yêu cầu: **PaymentSucceeded**, **PaymentFailed**, **BookingCancelled** (đã liệt kê ở trên).

Phần E - Chọn cách điều phối: đơn giản hay có người điều phối?

8. Với SmartBooking giai đoạn đầu, bạn chọn cách nào?

Chọn **kết hợp cả hai**, tùy theo từng phần của luồng:

- **Cách 2 (Mediator/Orchestrator)** cho phần lõi: **giữ chỗ** → **thanh toán** → **xác nhận/hủy booking**. Đây là "Booking Orchestrator" điều phối tuần tự và ra quyết định rẽ nhánh (thành công thì confirm, thất bại thì hủy).
- **Cách 1 (Broker)** cho các phần phụ trợ: **Notification**, **Report**, **Staff** tự lắng nghe sự kiện **BookingConfirmed/BookingCancelled** mà không cần ai điều phối.

9. Vì sao?

Luồng lõi (giữ chỗ → thanh toán → xác nhận) có yêu cầu nghiệp vụ chặt: phải biết chính xác thứ tự các bước, phải có khả năng "bù trừ" (compensating action - hủy giữ chỗ nếu thanh toán thất bại), nên hợp với Mediator topology - theo tài liệu, Mediator phù hợp khi "cần quản lý tập trung" và "xử lý logic phức tạp". Ngược lại, Notification và Report là các việc độc lập với nhau, không cần biết thứ tự thực hiện lẫn nhau, nên hợp với Broker topology - giúp giảm phụ thuộc, và sau này muốn thêm một kênh thông báo mới (vd Zalo, push notification) chỉ cần thêm một consumer mới lắng nghe sự kiện, không phải sửa vào phần lõi.

10. Nếu quy trình thanh toán phức tạp hơn, có nên dùng bộ phận điều phối không?

Có. Khi thanh toán có nhiều bước hơn (nhiều cổng thanh toán, xử lý trả góp, hoàn tiền một phần, cần retry/timeout...), việc theo dõi trạng thái từng bước và quyết định commit/compensate rất khó nếu để từng service tự phản ứng rời rạc (dễ mất đồng bộ trạng thái). Mediator giúp tập trung logic điều phối, dễ debug và đảm bảo tính nhất quán hơn.

11. Nếu chỉ gửi email và cập nhật báo cáo, có cần bộ phận điều phối không?

Không cần. Đây là hai việc độc lập, không có nhánh rẽ, không cần rollback lẫn nhau - dùng Broker (phát sự kiện, ai cần thì tự nghe) là đủ, tránh tạo thêm một điểm lỗi trung tâm (single point of failure) và độ phức tạp không cần thiết.

Phần F - Phân tích lỗi thực tế

1. Thanh toán thành công nhưng booking chưa được xác nhận

Hậu quả: Khách đã bị trừ tiền nhưng lịch hẹn vẫn ở trạng thái "chờ xác nhận" (do sự kiện `PaymentSucceeded` bị mất/chưa tới, hoặc Booking service tạm thời gặp sự cố) → khách hoang mang, gọi hỏi hỗ trợ, mất niềm tin vào hệ thống.

Cách xử lý đề xuất:

- Booking giữ slot ở trạng thái `Pending` kèm thời gian hết hạn (vd giữ chỗ 10 phút).
- Khi Booking nhận sự kiện `PaymentSucceeded`, dùng `paymentId` làm khóa idempotency để cập nhật sang `Confirmed` - nếu event bị gửi lại (duplicate) thì không xử lý lần hai.
- Cần một **job đối soát (reconciliation)** chạy định kỳ (vd mỗi 5 phút): so sánh các giao dịch `PaymentSucceeded` bên Payment với trạng thái booking tương ứng, phát hiện trường hợp lệch để tự động đồng bộ lại hoặc đẩy vào danh sách cho nhân viên xử lý tay.
- **Không tự động hủy booking** chỉ vì chưa nhận được event xác nhận kịp thời - vì tiền đã thu, hủy nhầm sẽ tệ hơn.
- Cần lưu trạng thái trung gian rõ ràng (vd `PaymentSucceeded_AwaitingConfirmation`) để không "mất dấu" ca lỗi này.

2. Gửi email/SMS bị lỗi

Hậu quả: Khách không nhận được xác nhận, có thể đến sai giờ hoặc tưởng lịch chưa được đặt, tăng số lượng cuộc gọi hỏi nhân viên.

Cách xử lý đề xuất:

- Booking đã thành công thì **không hủy booking** chỉ vì lỗi gửi thông báo - hai việc độc lập nhau.
- Notification retry theo cơ chế backoff, ví dụ **3 lần** (5 giây → 30 giây → 2 phút).
- Nếu sau 3 lần vẫn lỗi → chuyển trạng thái `NotificationFailed`, lưu vào danh sách lỗi để nhân viên kiểm tra/gửi thủ công, hoặc tự động chuyển sang kênh dự phòng (email lỗi thì thử gửi SMS).
- Cần idempotency key (`bookingId` + loại thông báo) để tránh gửi trùng khi các lần retry chồng lấn nhau.

3. Một sự kiện bị xử lý hai lần

Hậu quả: Ví dụ `BookingConfirmed` bị xử lý 2 lần → Report cộng dồn sai số lượng booking/doanh thu; hoặc Notification gửi email xác nhận 2 lần, gây khó chịu cho khách.

Cách xử lý đề xuất:

- Thiết kế consumer **idempotent**: mỗi event có `eventId` duy nhất, lưu vào bảng `processed_events` trước khi xử lý nghiệp vụ - nếu `eventId` đã tồn tại thì bỏ qua, không xử lý lại.

- Report nên dùng **upsert theo bookingId** thay vì cộng dồn (increment), để dù nhận trùng sự kiện cũng không sai số liệu.
- Không cần retry ở lỗi này (vấn đề là dư, không phải thiếu), nhưng bắt buộc phải có cơ chế chống trùng ở tầng consumer, vì hầu hết message broker chỉ đảm bảo "at-least-once delivery" - phải tự đảm bảo "effectively-once" bằng idempotency.

4. Số lượng người dùng tăng đột biến

Hậu quả: Booking service quá tải vào giờ cao điểm (vd khuyến mãi), phản hồi chậm/timeout khi kiểm tra lịch trống hoặc tạo booking; nếu không khóa đúng, có thể xảy ra double-booking khi nhiều request cùng chọn một khung giờ.

Cách xử lý đề xuất:

- Ở bước đồng bộ (kiểm tra lịch trống, tạo booking), dùng khóa mức dòng (row-level lock) hoặc optimistic locking trên slot để tránh double-booking khi có nhiều request cùng lúc.
- Các tác vụ phụ (thông báo, báo cáo, log) đẩy vào hàng đợi sự kiện - hàng đợi đóng vai trò bộ đệm (back-pressure), giúp hệ thống "hạ nhiệt" mà không mất dữ liệu khi traffic tăng đột biến.
- Có thể thêm rate-limiting ở tầng gateway và scale-out theo chiều ngang cho phần xử lý booking (vì Modular Monolith vẫn có thể chạy nhiều instance phía sau load balancer nếu thiết kế stateless).
- Không cần retry cụ thể ở lỗi này, nhưng cần giám sát (monitoring/alerting) để biết khi nào cần mở rộng tài nguyên, và đặt timeout hợp lý để tránh lỗi dây chuyền (cascading failure) sang các module khác.

Tổng kết

SmartBooking nên bắt đầu với **Modular Monolith**: đủ nhanh để kịp deadline 3 tháng với đội ngũ nhỏ, nhưng có ranh giới module rõ ràng (đặc biệt là Thanh toán và Thông báo) để sẵn sàng tách thành microservices khi công ty mở rộng chi nhánh sau 1 năm. Trong luồng đặt lịch, các bước ảnh hưởng trực tiếp đến người dùng (kiểm tra chỗ trống, tạo booking, thanh toán) xử lý đồng bộ ngay; các bước phụ trợ (thông báo, báo cáo, log) xử lý bằng sự kiện theo hướng Broker, riêng phần lõi giữ chỗ–thanh toán–xác nhận dùng một Mediator/Orchestrator nhỏ để đảm bảo tính nhất quán. Việc phân tích lỗi thực tế cho thấy: hầu hết rủi ro không nằm ở việc "chọn kiến trúc nào" mà ở cách xử lý trạng thái, retry, idempotency và đối soát dữ liệu khi có sự cố.

Ghi rõ đã dùng AI như thế nào

Bài làm có sử dụng AI:

- **Tra lại khái niệm:** hai tài liệu đọc xong vẫn còn vài chỗ chưa chắc, cần hỏi AI phân biệt rõ Monolith / Modular Monolith / Microservices khác nhau ở đâu trong thực tế, và Event, Broker, Mediator trong EDA cụ thể đóng vai trò gì trong một luồng xử lý.
- **Cho ví dụ để hình dung hơn:** vì tài liệu dùng ví dụ thương mại điện tử/gọi xe, phải nhờ AI diễn giải lại các ví dụ đó theo hướng gần với bài toán đặt lịch (SmartBooking) để dễ liên hệ khi làm Phần C, D, E.
- **Hỏi để phân biệt "xử lý ngay" và "xử lý bằng sự kiện":** hai khái niệm này chưa rõ khi điền bảng ở Phần C, phải hỏi AI cho tiêu chí phân biệt (việc nào người dùng cần thấy kết quả ngay thì xử lý đồng bộ, việc nào là phụ trợ/không cần real-time thì đẩy qua sự kiện), sau đó áp dụng tiêu chí đó vào từng bước của luồng đặt lịch.

- **Hoàn thiện bài làm:** Toàn bộ bài làm hiện tại đã được format lại bởi AI để dễ nhìn và trình bày mạch lạc hơn thay vì là bản nháp tự làm.